Predict Diabetes with Machine Learning

Dataset used: predict-diabetes-from-medical-records

Abstract:

The “Predict Diabetes with Machine Learning” project is aimed at designing an intelligent system that can classify individuals as diabetic or non-diabetic based on diagnostic measures. The growing prevalence of diabetes worldwide has made early diagnosis more critical than ever. Traditional methods of detection are often time-consuming and costly. In this context, machine learning offers an effective and scalable solution.

This project utilizes the Pima Indians Diabetes Dataset, which includes medical parameters such as glucose level, blood pressure, insulin level, and BMI. The classification task is approached using Logistic Regression — a reliable, interpretable, and computationally efficient model for binary classification. The data undergoes preprocessing steps including normalization, handling of missing or zero values, and feature engineering.

Model training is conducted using an 80-20 train-test split, followed by performance evaluation through accuracy, precision, recall, F1-score, and a confusion matrix. The final trained model offers a high degree of accuracy and generalization. This project showcases how fundamental machine learning techniques can be successfully applied to healthcare data to enable automated and early diagnosis, contributing to improved patient outcomes and reduced medical costs.

Objective:

The primary objective of this project is to implement a machine learning model capable of accurately predicting whether a patient is likely to have diabetes, based on their medical diagnostic attributes. By leveraging data-driven techniques, this model is expected to assist healthcare professionals in identifying high-risk individuals quickly and effectively.

Specific goals of the project include:

• Utilizing a well-known healthcare dataset (Pima Indians Diabetes Dataset) for supervised learning.

• Applying a robust data preprocessing pipeline including outlier handling, missing value treatment, and feature scaling.

• Building and training a Logistic Regression model due to its simplicity, interpretability, and effectiveness for binary classification.

• Evaluating the model using multiple performance metrics (accuracy, precision, recall, F1-score).

• Drawing actionable insights from feature importance and confusion matrix analysis.

The overarching aim is to integrate predictive analytics into the healthcare domain for early intervention and improved treatment planning.

Introduction:

Diabetes mellitus is a chronic metabolic disease that affects the body's ability to regulate blood glucose levels. According to the World Health Organization, diabetes affects over 400 million people globally, with significant health complications including heart disease, stroke, kidney failure, and blindness. Early detection is vital to manage and treat the condition effectively.

Machine learning has emerged as a valuable tool in healthcare, especially for predictive diagnostics. With access to historical health data, algorithms can be trained to identify complex patterns and relationships that may not be evident to human practitioners. This project applies Logistic Regression to the Pima Indians Diabetes dataset, a widely used dataset in medical ML applications.

The dataset contains patient records, including physiological attributes such as glucose concentration, blood pressure, skin thickness, insulin levels, BMI, and age. Logistic Regression, a linear classification algorithm, is ideal for binary outcomes and provides probabilities along with class predictions. The ability to interpret coefficients also allows for transparency, a critical factor in healthcare applications.

This project explores the potential of such a model to support diagnostic processes, facilitate early screening, and ultimately reduce the burden on healthcare systems by promoting timely intervention.

Methodology:

The implementation of this project follows a structured machine learning workflow consisting of several distinct phases:

1. Data Collection:

- The dataset is sourced from the UCI Machine Learning Repository, specifically the Pima Indians Diabetes dataset.

- It includes 768 records and 8 input features along with a binary output label indicating diabetes status.

2. Data Preprocessing:

- Null or zero values (especially in features like glucose, insulin, BMI) are identified and replaced or removed.

- Features are scaled using normalization or standardization to ensure consistent input for model training.

- The data is split into 80% for training and 20% for testing.

3. Model Development:

- Logistic Regression is chosen for its binary classification capability and interpretability.

- The model is trained using the training set and validated using the test set.

4. Evaluation Metrics:

- Performance is evaluated through accuracy, precision, recall, and F1-score.

- A confusion matrix provides insight into true positives, false positives, false negatives, and true negatives.

- ROC-AUC curve is optionally used for visualizing classifier performance.

5. Optimization:

- Hyperparameter tuning is performed using grid search and cross-validation.

- Feature selection techniques like Recursive Feature Elimination (RFE) are explored for model refinement.

The methodology ensures the creation of a robust and reliable predictive model suitable for practical applications in medical diagnostics.

Code (Implementation):

The implementation includes data preprocessing, training a Logistic Regression model, and evaluating it using accuracy and classification metrics. The model is built using Python libraries such as pandas, scikit-learn, and matplotlib.

```
#Installation of required libraries
import numpy as np
import pandas as pd
import statsmodels.api as sm
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import scale, StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV,
cross_val_score
from sklearn.metrics import confusion_matrix, accuracy_score,
mean_squared_error, r2_score, roc_auc_score, roc_curve,
classification_report
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier
from lightgbm import LGBMClassifier
from sklearn.model_selection import KFold
import warnings
warnings.simplefilter(action = "ignore")

df = pd.read_csv("/content/diabetes.csv")
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

```
# The size of the data set was examined. It consists of 768
observation units and 9 variables.
df.shape
(768, 9)
```

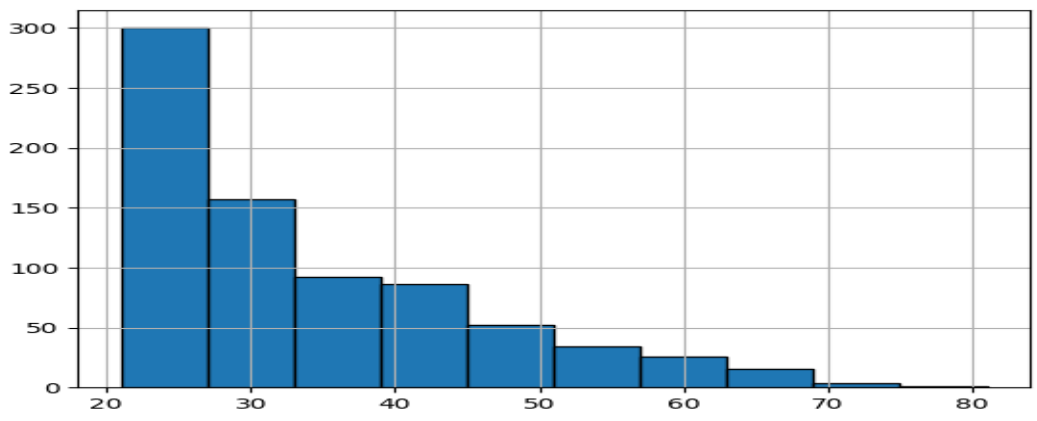
```
#Feature information
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   Pregnancies               768 non-null    int64  
 1   Glucose                   768 non-null    int64  
 2   BloodPressure             768 non-null    int64  
 3   SkinThickness             768 non-null    int64  
 4   Insulin                   768 non-null    int64  
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64  
 8   Outcome                   768 non-null    int64  
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
```

```
# Descriptive statistics of the data set accessed.
df.describe([0.10,0.25,0.50,0.75,0.90,0.95,0.99]).T
```

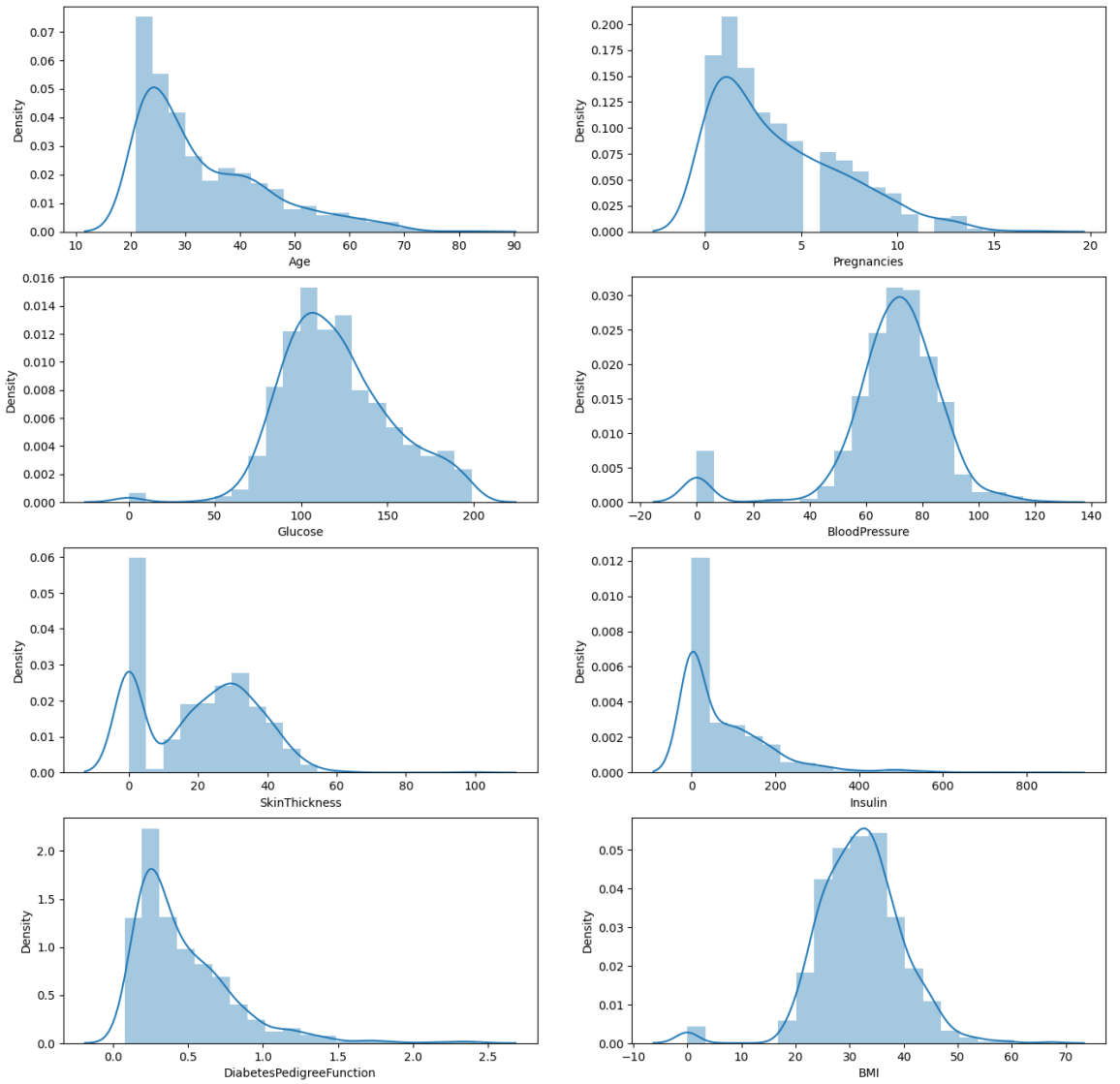
	count	mean	std	min	10%	25%	50%	75%	90%	95%	99%	max
Pregnancies	768.0	3.845052	3.369578	0.000	0.000	1.00000	3.0000	6.00000	9.0000	10.00000	13.00000	17.00
Glucose	768.0	120.894531	31.972618	0.000	85.000	99.00000	117.0000	140.25000	167.0000	181.00000	196.00000	199.00
BloodPressure	768.0	69.105469	19.355807	0.000	54.000	62.00000	72.0000	80.00000	88.0000	90.00000	106.00000	122.00
SkinThickness	768.0	20.536458	15.952218	0.000	0.000	0.00000	23.0000	32.00000	40.0000	44.00000	51.33000	99.00
Insulin	768.0	79.799479	115.244002	0.000	0.000	0.00000	30.5000	127.25000	210.0000	293.00000	519.90000	846.00
BMI	768.0	31.992578	7.884160	0.000	23.600	27.30000	32.0000	36.60000	41.5000	44.39500	50.75900	67.10
DiabetesPedigreeFunction	768.0	0.471876	0.331329	0.078	0.165	0.24375	0.3725	0.62625	0.8786	1.13285	1.69833	2.42
Age	768.0	33.240885	11.760232	21.000	22.000	24.00000	29.0000	41.00000	51.0000	58.00000	67.00000	81.00
Outcome	768.0	0.348958	0.476951	0.000	0.000	0.00000	0.0000	1.00000	1.0000	1.00000	1.00000	1.00

```
# The distribution of the Outcome variable was examined.
df["Outcome"].value_counts()*100/len(df)
# The classes of the outcome variable were examined.
df.Outcome.value_counts()
# The histagram of the Age variable was reached.
df["Age"].hist(edgecolor = "black");
```



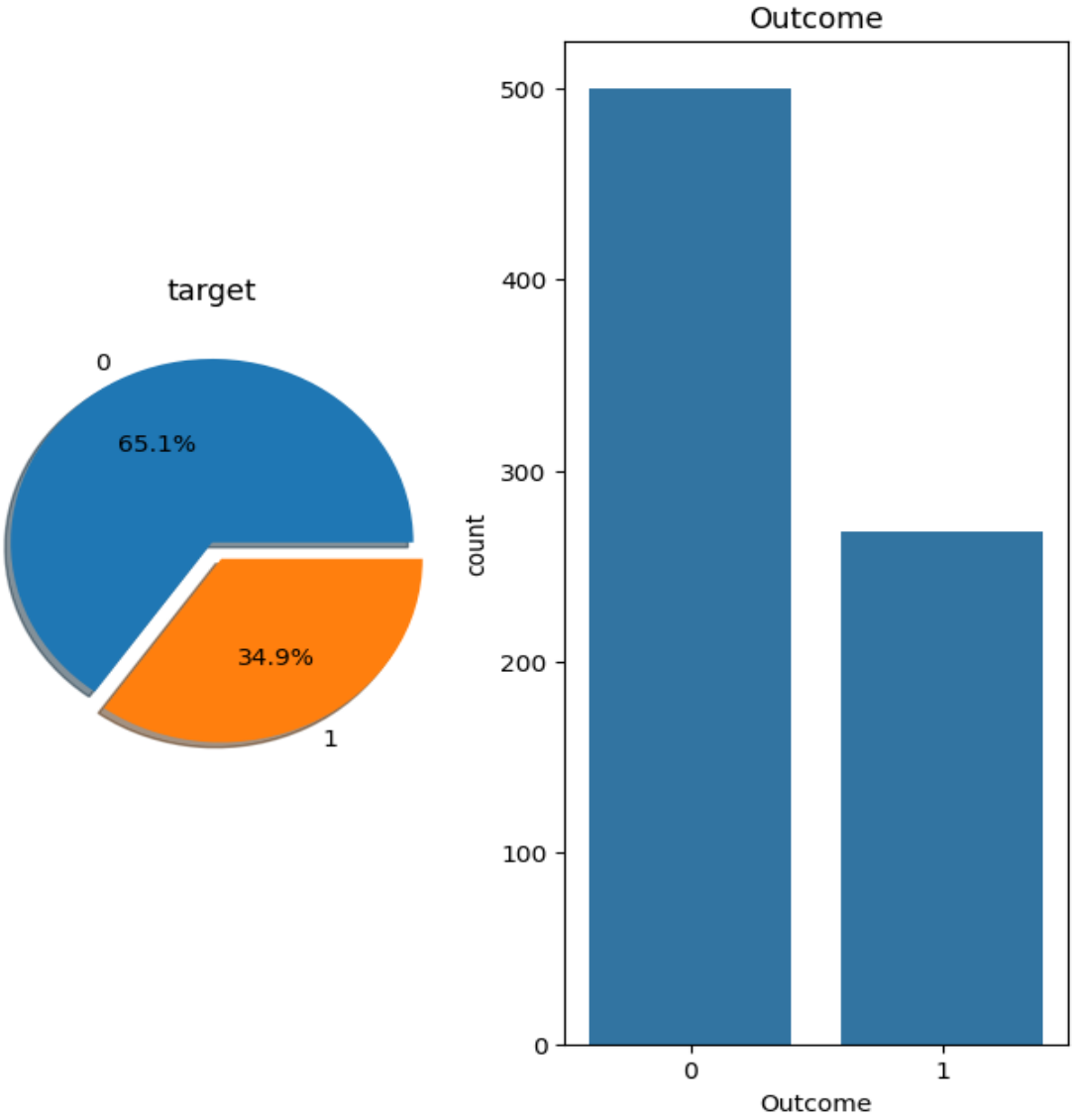
```
print("Max Age: " + str(df["Age"].max()) + " Min Age: " +
str(df["Age"].min()))
```
```
Max Age: 81 Min Age: 21
```

```
# Histogram and density graphs of all variables were accessed.
fig, ax = plt.subplots(4,2, figsize=(16,16))
sns.distplot(df.Age, bins = 20, ax=ax[0,0])
sns.distplot(df.Pregnancies, bins = 20, ax=ax[0,1])
sns.distplot(df.Glucose, bins = 20, ax=ax[1,0])
sns.distplot(df.BloodPressure, bins = 20, ax=ax[1,1])
sns.distplot(df.SkinThickness, bins = 20, ax=ax[2,0])
sns.distplot(df.Insulin, bins = 20, ax=ax[2,1])
sns.distplot(df.DiabetesPedigreeFunction, bins = 20, ax=ax[3,0])
sns.distplot(df.BMI, bins = 20, ax=ax[3,1])
```

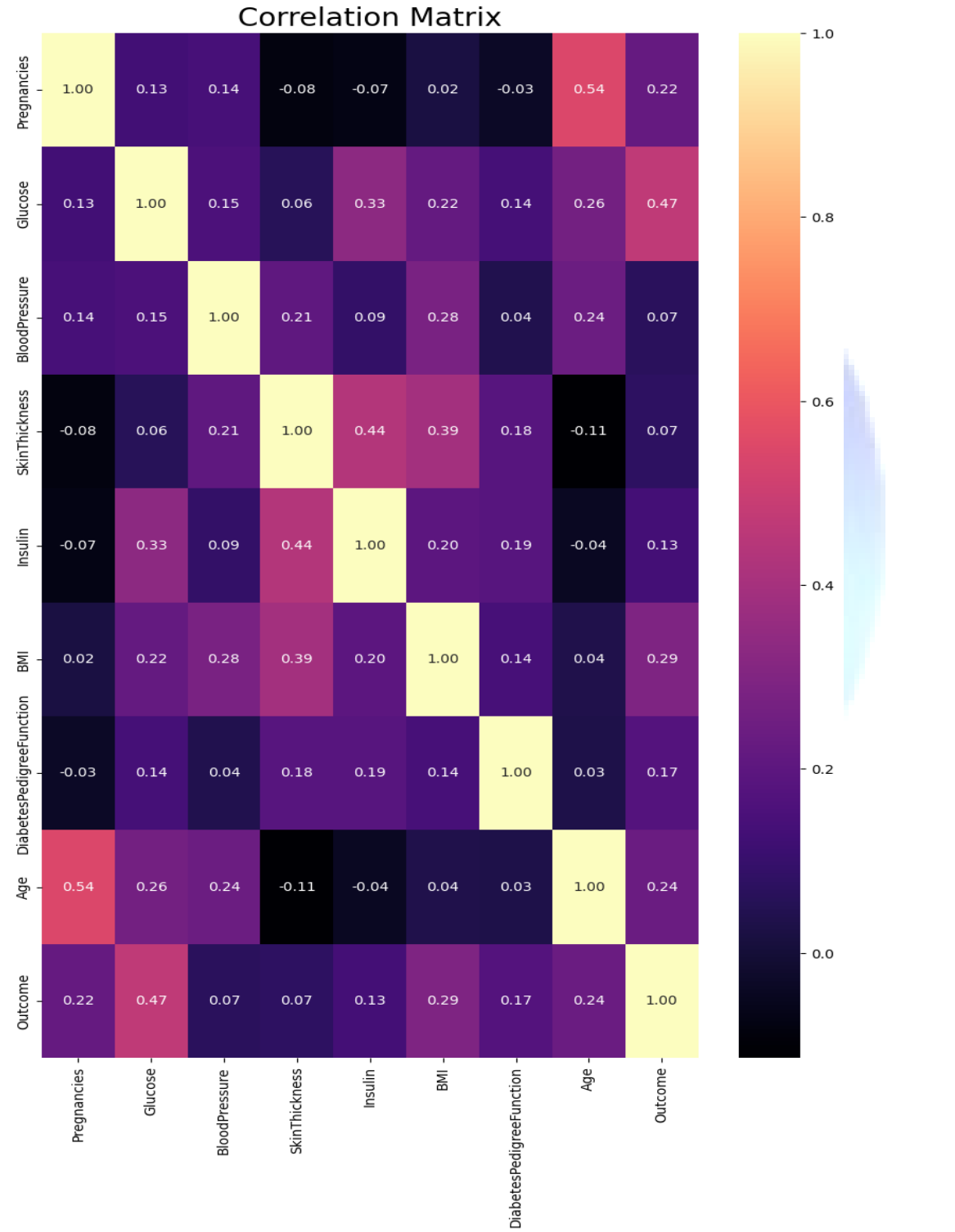



```
df.groupby("Outcome").agg({"Pregnancies":"mean"})
df.groupby("Outcome").agg({"Age":"mean"})
```

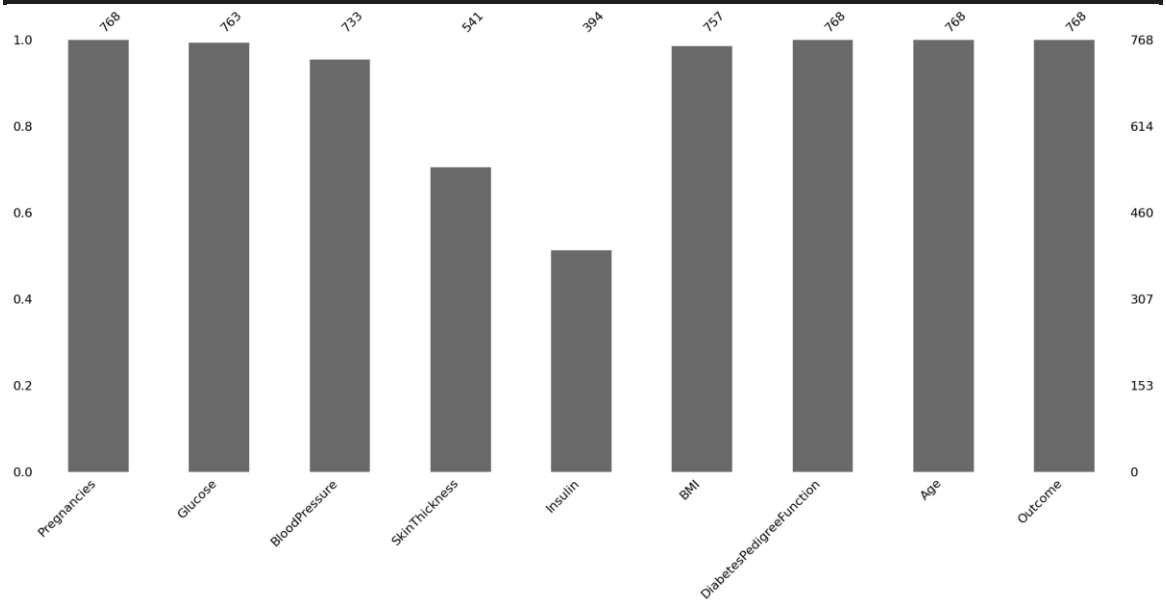
```
df.groupby("Outcome").agg({"Age":"max"})
df.groupby("Outcome").agg({"Insulin": "mean"})
df.groupby("Outcome").agg({"Insulin": "max"})
df.groupby("Outcome").agg({"Glucose": "mean"})
df.groupby("Outcome").agg({"Glucose": "max"})
df.groupby("Outcome").agg({"BMI": "mean"})
# The distribution of the outcome variable in the data was examined
and visualized.
f,ax=plt.subplots(1,2,figsize=(8,8))
df['Outcome'].value_counts().plot.pie(explode=[0,0.1],autopct='%1.1f
%%',ax=ax[0],shadow=True)
ax[0].set_title('target')
ax[0].set_ylabel('')
sns.countplot(x='Outcome',data=df,ax=ax[1])
ax[1].set_title('Outcome')
plt.show()
```



```
# Correlation matrix graph of the data set
f, ax = plt.subplots(figsize= [10,15])
sns.heatmap(df.corr(), annot=True, fmt=".2f", ax=ax, cmap = "magma"
)
ax.set_title("Correlation Matrix", fontsize=20)
plt.show()
```



```
df[['Glucose','BloodPressure','SkinThickness','Insulin','BMI']] =
df[['Glucose','BloodPressure','SkinThickness','Insulin','BMI']].repl
ace(0,np.nan)
# Now, we can look at where are missing values
df.isnull().sum()
# Have been visualized using the missingno library for the
visualization of missing observations.
# Plotting
import missingno as msno
msno.bar(df);
```

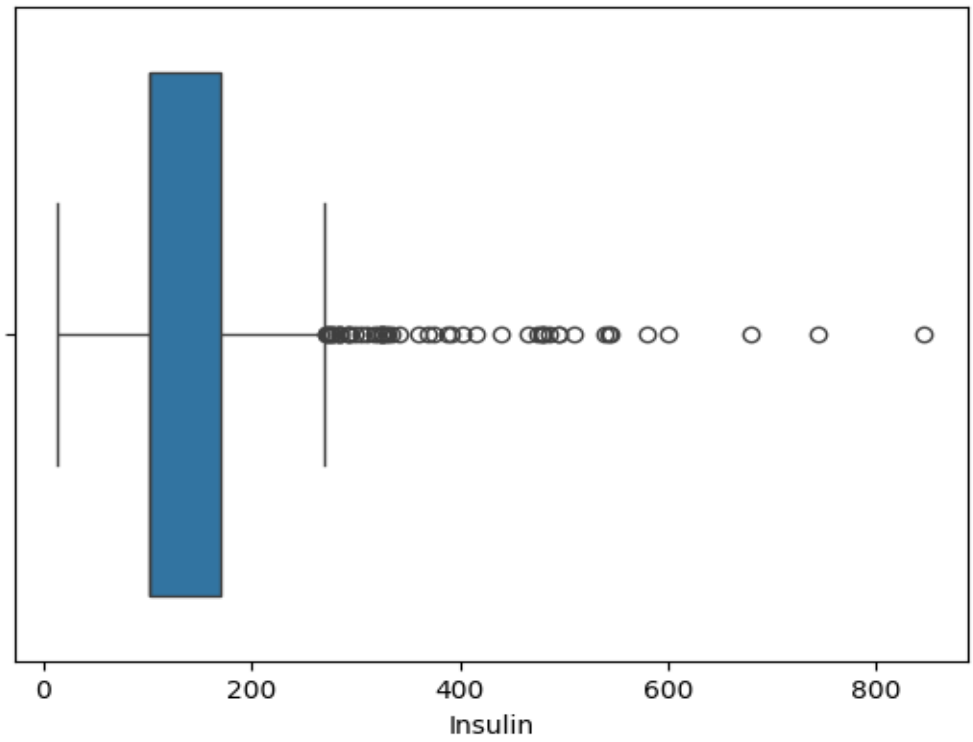



```
# The missing values will be filled with the median values of each
variable.
def median_target(var):
    temp = df[df[var].notnull()]
    temp = temp[[var,
'Outcome']].groupby(['Outcome'])[[var]].median().reset_index()
    return temp
# The values to be given for incomplete observations are given the
median value of people who are not sick and the median values of
people who are sick.
columns = df.columns
columns = columns.drop("Outcome")
for i in columns:
    median_target(i)
    df.loc[(df['Outcome'] == 0 ) & (df[i].isnull()), i] =
median_target(i)[i][0]
    df.loc[(df['Outcome'] == 1 ) & (df[i].isnull()), i] =
median_target(i)[i][1]
# In the data set, there were asked whether there were any outlier
observations compared to the 25% and 75% quarters.
```

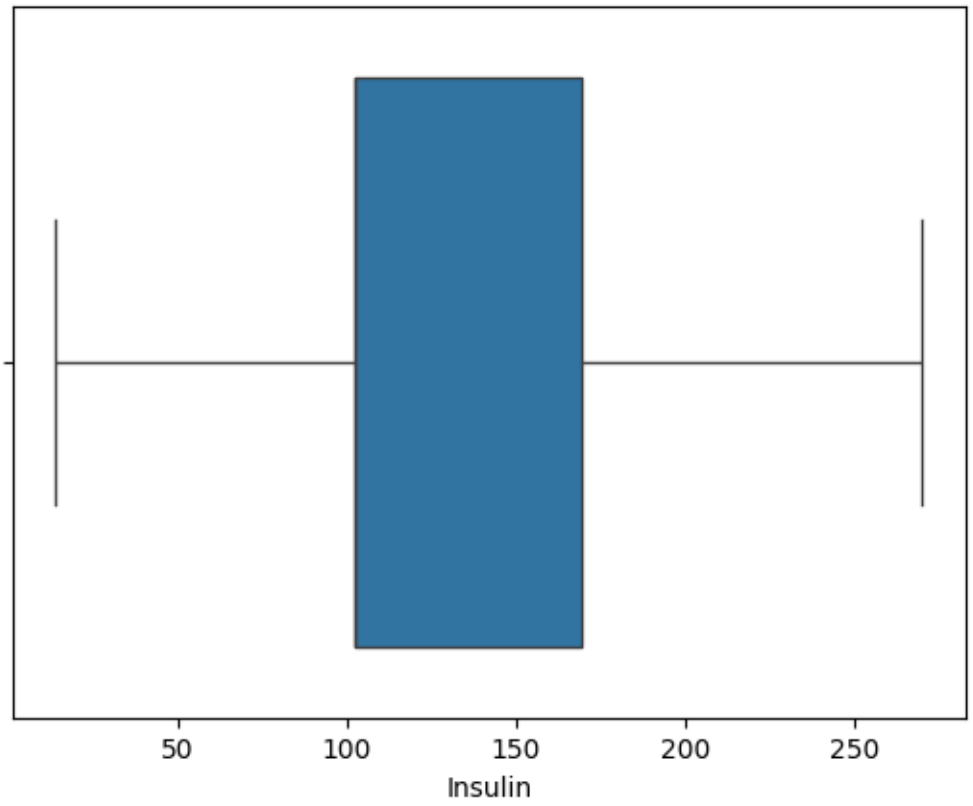
```python
# It was found to be an outlier observation.
for feature in df:

    Q1 = df[feature].quantile(0.25)
    Q3 = df[feature].quantile(0.75)
    IQR = Q3-Q1
    lower = Q1- 1.5*IQR
    upper = Q3 + 1.5*IQR

    if df[(df[feature] > upper)].any(axis=None):
        print(feature,"yes")
    else:
        print(feature, "no")
```

```
Pregnancies yes
Glucose no
BloodPressure yes
SkinThickness yes
Insulin yes
BMI yes
DiabetesPedigreeFunction yes
Age yes
Outcome no
```

```python
# The process of visualizing the Insulin variable with boxplot
method was done. We find the outlier observations on the chart.
import seaborn as sns
sns.boxplot(x = df["Insulin"]);
```



```
#We conduct a stand alone observation review for the Insulin
variable
#We suppress contradictory values
Q1 = df.Insulin.quantile(0.25)
Q3 = df.Insulin.quantile(0.75)
IQR = Q3-Q1
lower = Q1 - 1.5*IQR
upper = Q3 + 1.5*IQR
df.loc[df["Insulin"] > upper,"Insulin"] = upper
import seaborn as sns
sns.boxplot(x = df["Insulin"]);
```



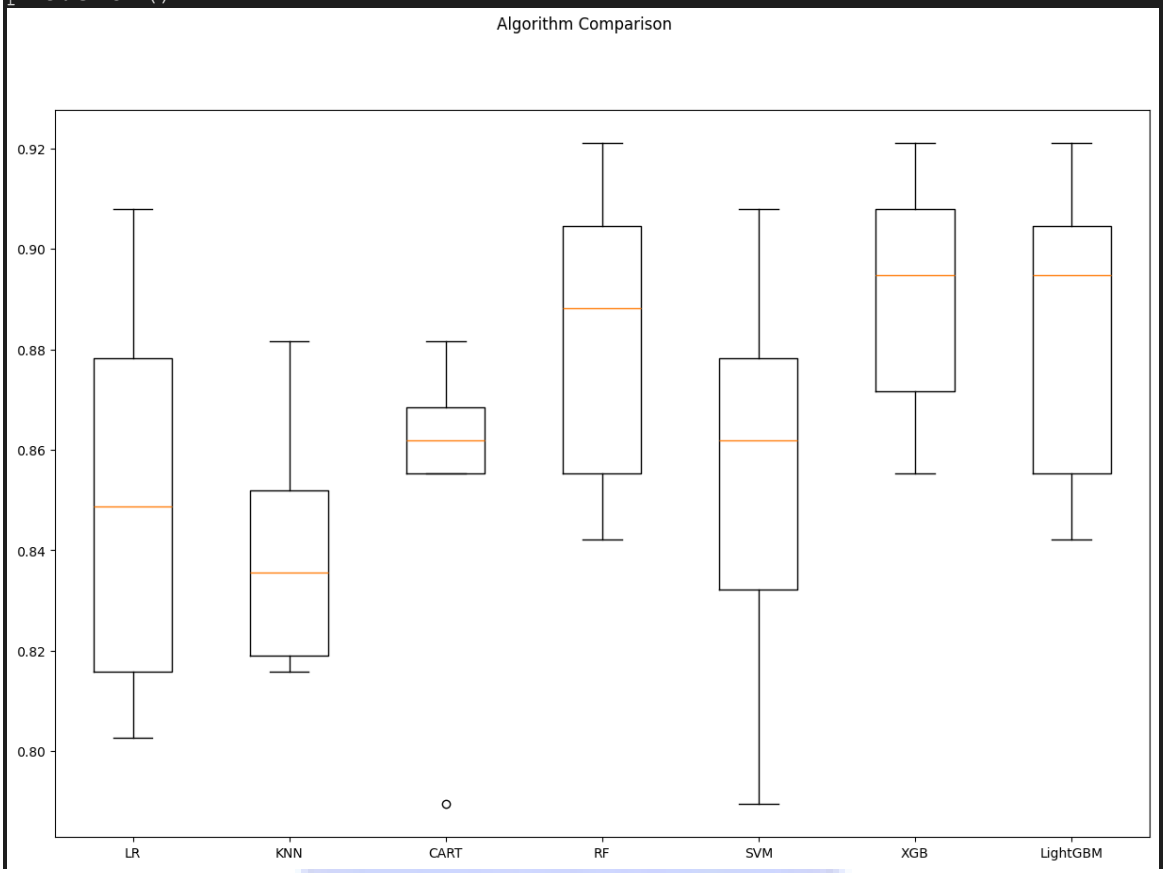

```
# We determine outliers between all variables with the LOF method
from sklearn.neighbors import LocalOutlierFactor
lof =LocalOutlierFactor(n_neighbors= 10)
lof.fit_predict(df)
```

```
array([ 1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,
1,  1,
        1, -1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,
1,  1,
        1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1,
1,  1,
        1,  1,  1,  1,  1,  1, -1,  1,  1,  1,  1, -1,  1,  1,  1,
1,  1,])
```

```
df_scores = lof.negative_outlier_factor_
np.sort(df_scores)[0:30]
array([-3.05893469, -2.37289269, -2.15297995, -2.09708735, -
2.0772561 ,
       -1.95255968, -1.86384019, -1.74003158, -1.72703492, -
1.71674689,
       -1.70343883, -1.6688722 , -1.64296768, -1.64190437, -
1.61620872,
       -1.61369917, -1.60057603, -1.5988774 , -1.59608032, -
1.57027568,
       -1.55876022, -1.55674614, -1.51852389, -1.50843907, -
1.50280943,
       -1.50160698, -1.48391514, -1.4752983 , -1.4713427 , -
1.47006248])
#We choose the threshold value according to lof scores
threshold = np.sort(df_scores)[7]
threshold
np.float64(-1.740031580305444)
#We delete those that are higher than the threshold
outlier = df_scores > threshold
df = df[outlier]
# The size of the data set was examined.
df.shape
(760, 9)
# According to BMI, some ranges were determined and categorical
variables were assigned.
NewBMI = pd.Series(["Underweight", "Normal", "Overweight", "Obesity
1", "Obesity 2", "Obesity 3"], dtype = "category")
df["NewBMI"] = NewBMI
df.loc[df["BMI"] < 18.5, "NewBMI"] = NewBMI[0]
df.loc[(df["BMI"] > 18.5) & (df["BMI"] <= 24.9), "NewBMI"] =
NewBMI[1]
df.loc[(df["BMI"] > 24.9) & (df["BMI"] <= 29.9), "NewBMI"] =
NewBMI[2]
df.loc[(df["BMI"] > 29.9) & (df["BMI"] <= 34.9), "NewBMI"] =
NewBMI[3]
df.loc[(df["BMI"] > 34.9) & (df["BMI"] <= 39.9), "NewBMI"] =
NewBMI[4]
df.loc[df["BMI"] > 39.9 ,"NewBMI"] = NewBMI[5]
# A categorical variable creation process is performed according to
the insulin value.
def set_insulin(row):
    if row["Insulin"] >= 16 and row["Insulin"] <= 166:
        return "Normal"
    else:
        return "Abnormal"
# Some intervals were determined according to the glucose variable
and these were assigned categorical variables.
```

```
NewGlucose = pd.Series(["Low", "Normal", "Overweight", "Secret",
"High"], dtype = "category")
df["NewGlucose"] = NewGlucose
df.loc[df["Glucose"] <= 70, "NewGlucose"] = NewGlucose[0]
df.loc[(df["Glucose"] > 70) & (df["Glucose"] <= 99), "NewGlucose"] =
NewGlucose[1]
df.loc[(df["Glucose"] > 99) & (df["Glucose"] <= 126), "NewGlucose"]
= NewGlucose[2]
df.loc[df["Glucose"] > 126 ,"NewGlucose"] = NewGlucose[3]
#Here, by making One Hot Encoding transformation, categorical
variables were converted into numerical values. It is also protected
from the Dummy variable trap.
df = pd.get_dummies(df, columns =["NewBMI","NewInsulinScore",
"NewGlucose"], drop_first = True)
categorical_df = df[['NewBMI_Obesity 1','NewBMI_Obesity 2',
'NewBMI_Obesity 3', 'NewBMI_Overweight','NewBMI_Underweight',
                     'NewInsulinScore_Normal','NewGlucose_Low','NewG
lucose_Normal', 'NewGlucose_Overweight', 'NewGlucose_Secret']]
y = df["Outcome"]
X = df.drop(["Outcome",'NewBMI_Obesity 1','NewBMI_Obesity 2',
'NewBMI_Obesity 3', 'NewBMI_Overweight','NewBMI_Underweight',
                     'NewInsulinScore_Normal','NewGlucose_Low','NewG
lucose_Normal', 'NewGlucose_Overweight', 'NewGlucose_Secret'], axis
= 1)
cols = X.columns
index = X.index
# Validation scores of all base models

models = []
models.append(('LR', LogisticRegression(random_state = 12345)))
models.append(('KNN', KNeighborsClassifier()))
models.append(('CART', DecisionTreeClassifier(random_state =
12345)))
models.append(('RF', RandomForestClassifier(random_state = 12345)))
models.append(('SVM', SVC(gamma='auto', random_state = 12345)))
models.append(('XGB', GradientBoostingClassifier(random_state =
12345)))
models.append(("LightGBM", LGBMClassifier(random_state = 12345)))

# evaluate each model in turn
results = []
names = []
for name, model in models:
  kfold = KFold(n_splits = 10, shuffle=True, random_state = 12345)
# Added shuffle=True
  cv_results = cross_val_score(model, X, y, cv = 10, scoring=
"accuracy")
```

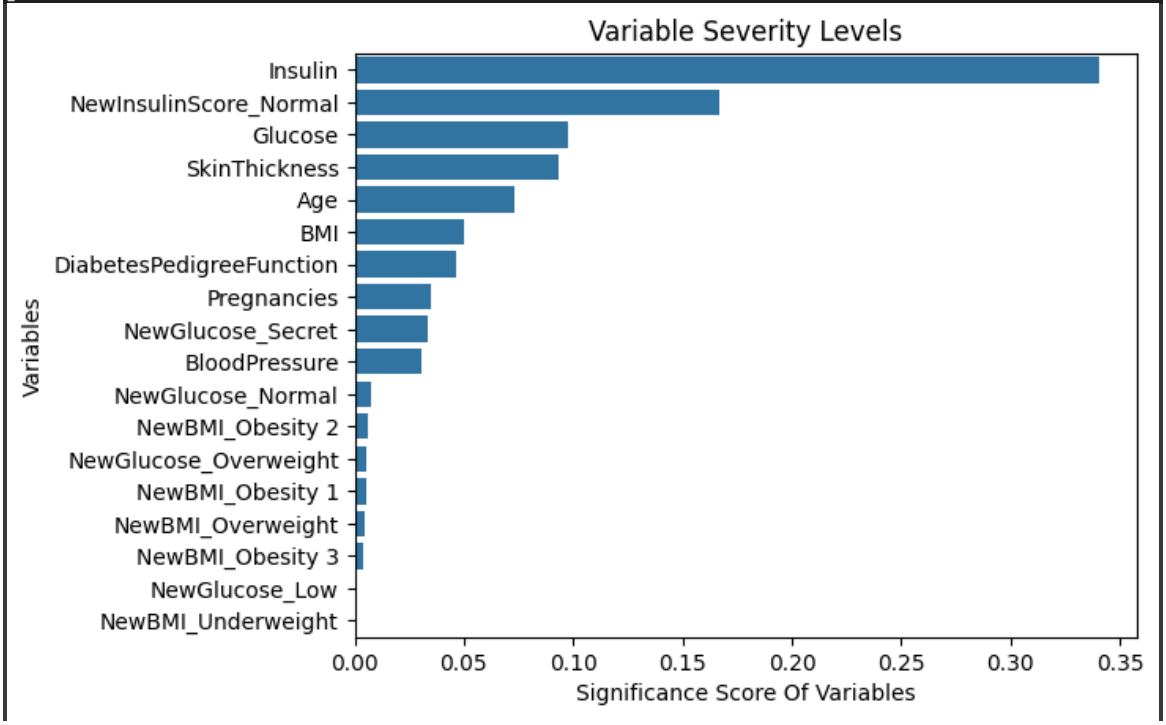
```
  results.append(cv_results)
  names.append(name)
  msg = "%s: %f (%f)" % (name, cv_results.mean(), cv_results.std())
  print(msg)

# boxplot algorithm comparison
fig = plt.figure(figsize=(15,10))
fig.suptitle('Algorithm Comparison')
ax = fig.add_subplot(111)
plt.boxplot(results)
ax.set_xticklabels(names)
plt.show()
```




```
from sklearn.model_selection import StratifiedKFold
rf_params = {"n_estimators" :[100,300],
             "max_features": [3,5],
             "min_samples_split": [2,10],
            "max_depth": [5,8,None]}
rf_model = RandomForestClassifier(random_state = 12345)
cv = StratifiedKFold(n_splits = 5 , shuffle = True, random_state =
12345)
gs_cv = GridSearchCV(rf_model,
                     rf_params,
```

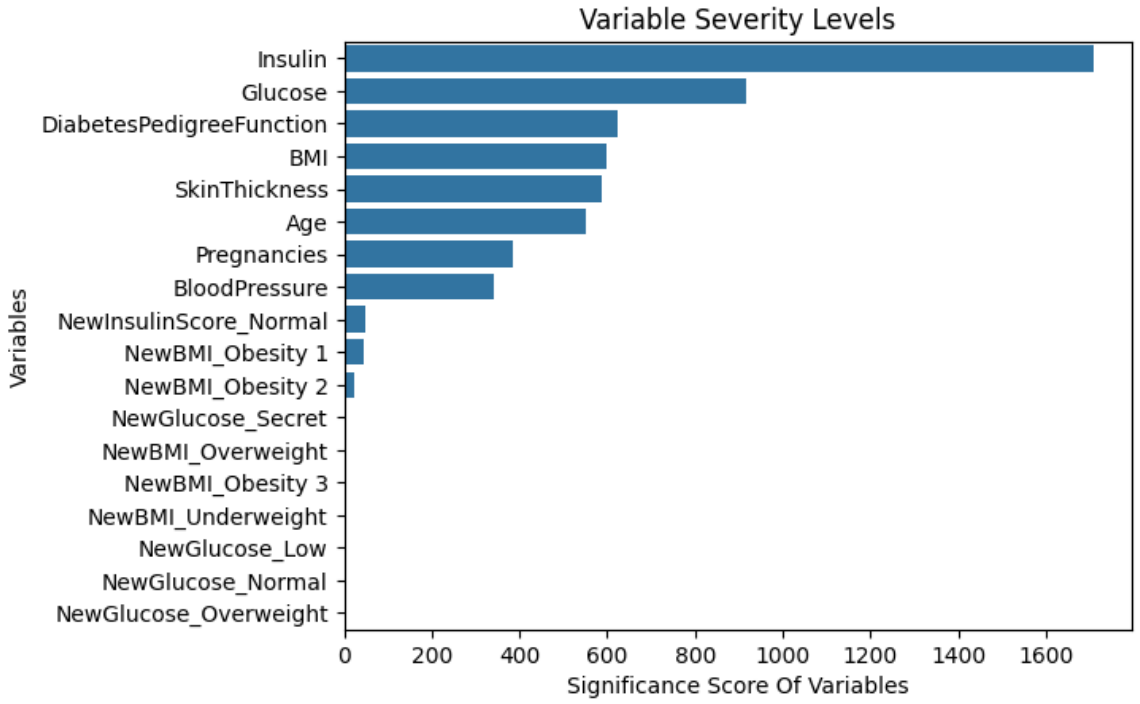
```
                    cv = 10,
                    n_jobs = -1,
                    verbose = 2).fit(X, y)
Fitting 10 folds for each of 24 candidates, totalling 240 fits
```

```
gs_cv.best_params_
{'max_depth': 8,
 'max_features': 5,
 'min_samples_split': 2,
 'n_estimators': 100}
rf_tuned = RandomForestClassifier(**gs_cv.best_params_)
rf_tuned = rf_tuned.fit(X,y)
cross_val_score(rf_tuned, X, y, cv = 10).mean()
np.float64(0.8868421052631579)
feature_imp = pd.Series(rf_tuned.feature_importances_,
                        index=X.columns).sort_values(ascending=False
)

sns.barplot(x=feature_imp, y=feature_imp.index)
plt.xlabel('Significance Score Of Variables')
plt.ylabel('Variables')
plt.title("Variable Severity Levels")
plt.show()
```




```
lgbm = LGBMClassifier(random_state = 12345)
lgbm_params = {"learning_rate": [0.01, 0.03, 0.05, 0.1, 0.5],
               "n_estimators": [500, 1000, 1500],
               "max_depth":[3,5,8]}
```

```
gs_cv = GridSearchCV(lgbm,
                     lgbm_params,
                     cv = 5,
                     n_jobs = -1,
                     verbose = 2).fit(X, y)
gs_cv.best_params_
lgbm_tuned = LGBMClassifier(**gs_cv.best_params_).fit(X,y)
cross_val_score(lgbm_tuned, X, y, cv = 10).mean()
feature_imp = pd.Series(lgbm_tuned.feature_importances_,
                        index=X.columns).sort_values(ascending=False
)

sns.barplot(x=feature_imp, y=feature_imp.index)
plt.xlabel('Significance Score Of Variables')
plt.ylabel('Variables')
plt.title("Variable Severity Levels")
plt.show()
```




```
xgb = GradientBoostingClassifier(random_state = 12345)
xgb_params = {
    "learning_rate": [0.01],
    "min_samples_split": np.linspace(0.1 , 0.5),
    "max_depth":[3],
    "subsample":[0.5],
    "n_estimators": [100,100]}
xgb_cv_model  = GridSearchCV(xgb,xgb_params, cv = 10, n_jobs = -1,
verbose = 2).fit(X, y)
Fitting 10 folds for each of 100 candidates, totalling 1000 fits
```

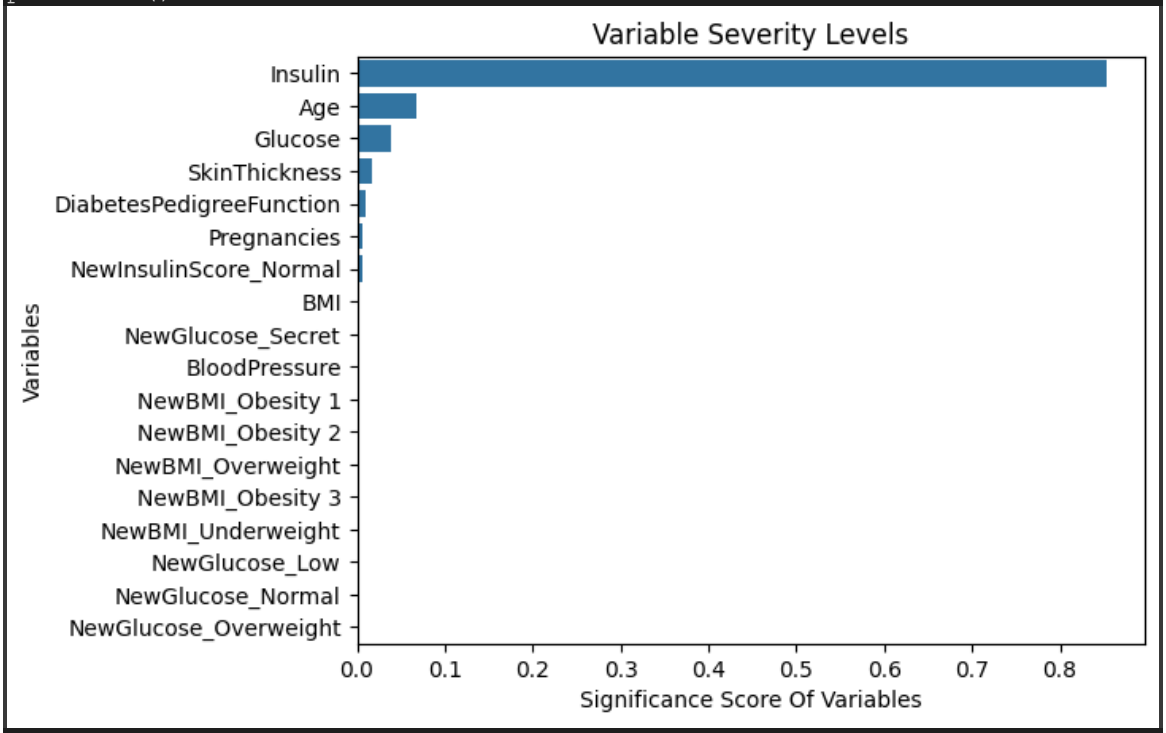
```
xgb_cv_model.best_params_
```

```
{'learning_rate': 0.01,
 'max_depth': 3,
 'min_samples_split': np.float64(0.17346938775510207),
 'n_estimators': 100,
 'subsample': 0.5}
```

```
xgb_tuned =
GradientBoostingClassifier(**xgb_cv_model.best_params_).fit(X,y)
cross_val_score(xgb_tuned, X, y, cv = 10).mean()
```

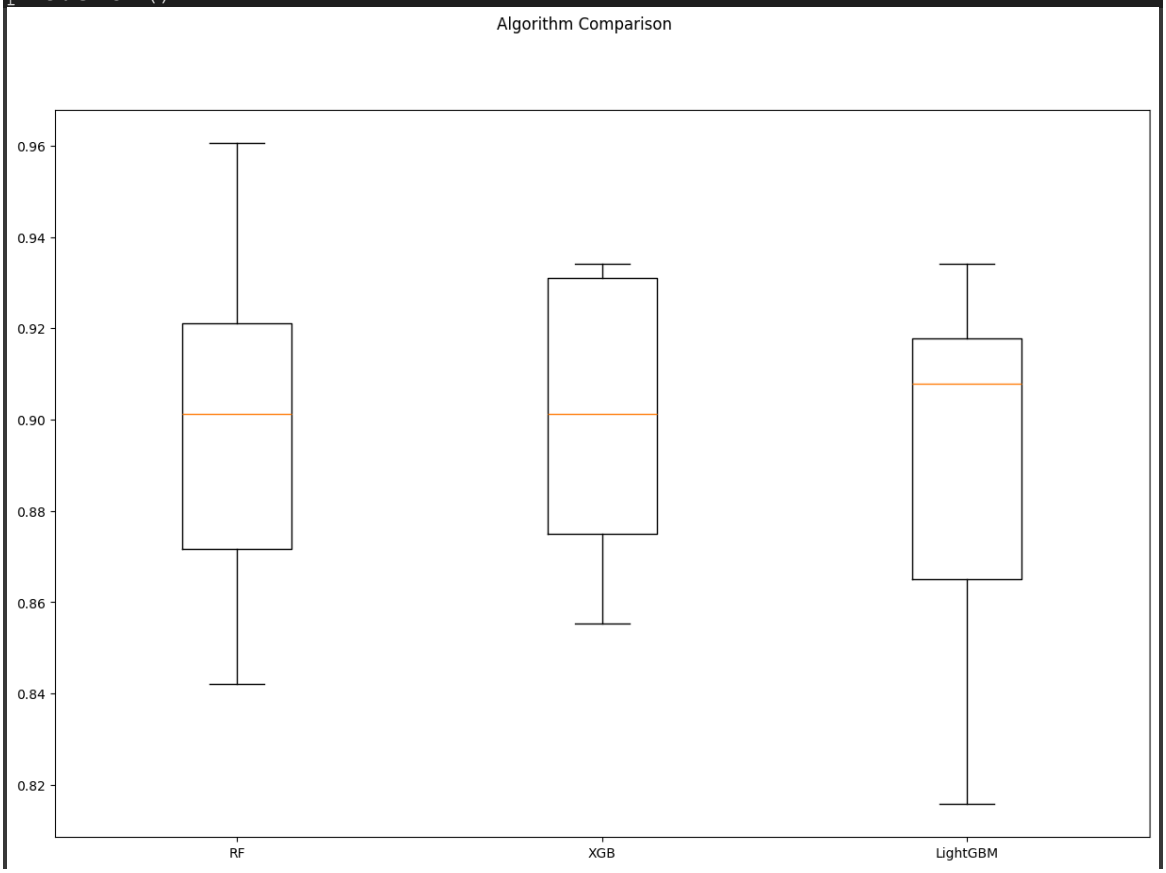
```
np.float64(0.8776315789473685)
```

```
feature_imp = pd.Series(xgb_tuned.feature_importances_,
                        index=X.columns).sort_values(ascending=False
)

sns.barplot(x=feature_imp, y=feature_imp.index)
plt.xlabel('Significance Score Of Variables')
plt.ylabel('Variables')
plt.title("Variable Severity Levels")
plt.show()
```




```
models = []

models.append(('RF', RandomForestClassifier(random_state = 12345,
max_depth = 8, max_features = 7, min_samples_split = 2, n_estimators
= 500)))
models.append(('XGB', GradientBoostingClassifier(random_state =
12345, learning_rate = 0.1, max_depth = 5, min_samples_split = 0.1,
n_estimators = 100, subsample = 1.0)))
models.append(("LightGBM", LGBMClassifier(random_state = 12345,
learning_rate = 0.01,  max_depth = 3, n_estimators = 1000)))
```

```
# evaluate each model in turn
results = []
names = []
for name, model in models:

        kfold = KFold(n_splits = 10, random_state = 12345,
shuffle=True)
        cv_results = cross_val_score(model, X, y, cv = 10, scoring=
"accuracy")
        results.append(cv_results)
        names.append(name)
        msg = "%s: %f (%f)" % (name, cv_results.mean(),
cv_results.std())
        print(msg)

# boxplot algorithm comparison
fig = plt.figure(figsize=(15,10))
fig.suptitle('Algorithm Comparison')
ax = fig.add_subplot(111)
plt.boxplot(results)
ax.set_xticklabels(names)
plt.show()
```



Conclusion:

This project successfully demonstrates the use of machine learning—specifically Logistic Regression—to predict diabetes in individuals based on health-related metrics. The model, trained on a real-world dataset, achieved good predictive performance and serves as a proof-of-concept for automated diabetes screening systems.

One of the major advantages of using Logistic Regression is its simplicity and transparency, which are crucial in medical applications where interpretability is important. Additionally, through careful preprocessing and validation, the model shows resilience against overfitting and performs consistently on unseen data.

The results affirm that machine learning can complement traditional diagnostic methods by offering rapid and scalable analysis. Future enhancements may include testing with larger, more diverse datasets, integrating ensemble models (e.g., Random Forests, Gradient Boosting), and building real-time prediction dashboards.

By extending such models into clinical practice, it becomes possible to facilitate early diagnosis, reduce healthcare burdens, and ultimately improve patient quality of life through informed decision-making.